Building a "chat with a PDF" wrapper in 20 lines of LangChain will not impress hiring managers. Employers hiring for AI and full-stack engineering roles look for engineers who understand why naive RAG fails in production—and how to solve **chunking degradation, exact-match blind spots, permission leaks, hallucinations, and evaluation blindness**.

The Project: "OpsPulse" — Multi-Tenant Engineering Runbook & Incident Triage Engine

You will build an internal knowledge assistant for an engineering organization. It ingests heterogeneous technical documentation (Markdown architecture docs, post-mortem PDFs, and JSON/CSV incident tickets) and answers developer queries with strict inline citations, role-based access control (RBAC), and verifiable evaluation metrics.

Dimension	Tutorial RAG (What to Avoid)	Production RAG (What You Will Build)
Ingestion	Synchronous upload blocking the HTTP thread	Asynchronous background queue with idempotency and content hashing
Chunking	Fixed 500-character split (breaks tables and code blocks)	Structure-aware splitting (Markdown headers/AST) + Contextual Retrieval
Retrieval	Vector similarity only (fails on error codes like ERR-4523)	Hybrid Search (Dense Vectors + BM25 Keyword) merged via Reciprocal Rank Fusion
Precision	Top-5 cosine similarity passed directly to LLM	Two-stage pipeline: Retrieve Top-30 candidates → Cross-Encoder Rerank to Top-5
Security	Prompting the LLM: *"Do not show admin data to guests"*	Hard metadata filtering at the database query layer before retrieval
Quality Control	"Vibe checking" answers manually in a chat UI	Automated CI evaluation suite measuring Faithfulness and Context Recall

The Hard Rules (Non-Negotiable Constraints)

Treat these rules as architectural guardrails. Violating any of them defeats the learning objective of the project:

1. **No High-Level "Black-Box" Chains:** You are banned from using one-line abstractions like RetrievalQA.from_chain_type() or VectorStoreIndex.from_documents(). You must write the ingestion, chunking, SQL/vector querying, reranking, and prompt construction layers explicitly using raw SDKs (OpenAI, Anthropic, or local Ollama SDKs) so you can inspect every SQL query and prompt payload.
2. **Never Enforce Permissions in the Prompt:** Every document must have an access role (public, engineer, admin). Permission filtering **must** happen inside the WHERE clause or vector metadata filter at retrieval time—never by asking the LLM to withhold sensitive chunks.
3. **No Fixed-Character Chunking:** Splitting text arbitrarily every N characters is forbidden.

You must split by semantic document structure (headings, paragraphs, intact code blocks, and intact tables).

4. **Mandatory Structured Outputs & Verifiable Citations:** The LLM must return a strict schema (using Pydantic, Zod, or native JSON Schema mode) containing answer, citations (an array of exact chunk_id and source_file pairs), and confidence_score. Every factual claim in the answer must reference a retrieved chunk_id.
5. **Deterministic Refusal Policy:** If the reranker's highest relevance score falls below a defined threshold, or if the retrieved context does not contain the answer, your system must deterministically abstain (*"Insufficient evidence in the knowledge base"*) rather than allowing the LLM to guess.
6. **No Pipeline Tweaks Without Benchmark Runs:** Once your baseline works, you may not change chunk sizes, prompts, or embedding models without running your automated evaluation script and logging the before-and-after metrics.

Project Tasks & Implementation Roadmap

1. **Build the Corpus & Evaluation Harness First**
   Prerequisite - Do not tune retrieval without a baseline benchmark
   In production, you cannot improve what you cannot measure. Before writing complex retrieval logic, assemble a corpus of 20 to 30 technical documents (use open-source documentation like Kubernetes runbooks, PostgreSQL docs, or realistic synthetic post-mortems containing specific error codes, CLI flags, and architecture tables).
   - **What to build:** Create a "Golden Dataset" JSON file containing 30 test cases. Each test case needs a question, expected_answer, expected_source_docs, and user_role. Include tricky queries: exact error codes (OOMKilled exit code 137), multi-hop questions spanning two files, and unanswerable questions designed to test refusal.
   - **Wire up the Eval CLI:** Build a script that runs your pipeline against the 30 questions and uses an LLM-as-a-Judge framework to score **Context Recall** (did you fetch the right chunk?), **Context Precision** (did you rank relevant chunks at the top?), and **Faithfulness** (did the LLM stick strictly to the context without hallucinating?).
   - **Direction & Sources:**
     - Read the **RAGAS Documentation** (docs.ragas.io) on Core Metrics (*Faithfulness*, *Context Precision*, *Context Recall*).
     - Alternatively, explore **Promptfoo** (promptfoo.dev/docs/guides/evaluate-rag/) to run declarative YAML-based RAG evaluations directly in your terminal and CI pipeline.
2. **Async Ingestion & Contextual Structure-Aware Chunking**
   Supports Markdown, PDFs, and Tables without losing parent context
   Build an asynchronous ingestion worker (using a background job queue in your preferred stack) that extracts clean Markdown from raw files, splits them by structural boundaries, and enriches each chunk before embedding.
   - **What to build:**
     1. **Document Parser:** Convert incoming PDFs and docs into structured Markdown, preserving tables and code blocks intact.
     2. **Heading-Aware Chunker:** Split documents by Markdown headers (#, ##,

###) so each chunk stays between 250 and 600 tokens while keeping code blocks unbroken. Store the parent heading hierarchy (H1 → H2 → H3), source_file, updated_at, and access_role in the chunk's metadata. Add a 10% to 15% token overlap between sequential chunks within the same section.

3. **Contextual Retrieval Enrichment:** A chunk that says *"Increase the timeout to 30s"* is useless if separated from its document title. Before creating the embedding and keyword index, pass the whole document (utilizing prompt caching to keep costs tiny) and the chunk to a fast, cheap LLM to prepend a 1-to-2 sentence context header explaining what document and system the chunk belongs to.
4. **Idempotency:** Hash each document's content (SHA-256). Re-ingesting an unchanged file must be a no-op; ingesting an updated file must atomically replace its old chunks.

- **Direction & Sources:**
  - Study **Anthropic's Engineering Guide on Contextual Retrieval** (anthropic.com/news/contextual-retrieval) to understand how prepending chunk context + prompt caching cuts retrieval failure rates by nearly 50%.
  - Use **Docling** (github.com/DS4SD/docling by IBM) for converting complex PDFs and tables into clean Markdown.

3. **Hybrid Search Engine & RBAC Filtering**
   Combines dense vector semantics with exact keyword matching
   Pure vector search fails when an engineer searches for a specific function name, ticket ID (INC-8842), or hex code. You must combine dense embeddings with lexical search (BM25 / Full-Text Search).
   - **What to build:**
     1. **Storage Layer:** Set up PostgreSQL with pgvector (or Qdrant). In PostgreSQL, create a table storing the raw chunk text, the contextualized text, a JSONB metadata column (with a GIN index), a vector column (with an HNSW index), and a tsvector column for full-text keyword search.
     2. **Dual Retrieval with RBAC:** Write a retrieval service that accepts (query, user_role). Execute both a vector cosine similarity search and a BM25/full-text keyword search—both strictly filtered by access_role in the database query.
     3. **Reciprocal Rank Fusion (RRF):** Merge the top 25 vector results and top 25 keyword results using the RRF formula: \text{RRF_Score}(d) = \sum_{m \in \{\text{vec}, \text{bm25}\}} \frac{1}{k + \text{rank}_m(d)} where k = 60 is a standard smoothing constant.
   - **Direction & Sources:**
     - Check **pgvector's Official Hybrid Search Examples** (github.com/pgvector/pgvector) or **Supabase's Hybrid Search SQL Guide** (supabase.com/docs/guides/ai/hybrid-search) for implementing RRF directly in SQL or application code.
     - If using a dedicated vector DB, read **Qdrant's Hybrid Queries & RRF**

Documentation (qdrant.tech/documentation/concepts/hybrid-queries/).

4. **Query Rewriting & Cross-Encoder Reranking**
   Two-stage retrieval pipeline for high precision
   Bi-encoder embeddings compress an entire chunk into a single vector, losing fine-grained relationship details between the user's question and the text. Reranking solves this by scoring the query and candidate chunks together.
   - **What to build:**
     1. **Conversational Query Rewriter:** Users ask follow-up questions with pronouns (e.g., *"How do I roll it back?"*). Build a lightweight pre-processing step that takes the chat history + latest message and rewrites it into a standalone, search-optimized query (*"How do I roll back the payment-service deployment in production?"*) before hitting the database.
     2. **Cross-Encoder Reranker:** Take the Top-25 merged candidates from your Hybrid RRF step and pass them through a Cross-Encoder Reranker. Select only the Top-5 chunks that exceed your minimum confidence threshold to send to the final LLM prompt.
     3. **Benchmark Comparison:** Run your Step 1 Evaluation CLI comparing *Vector-Only* vs. *Hybrid + Reranking*. Document the jump in Context Precision and Recall in your README.md.
   - **Direction & Sources:**
     - Use **Cohere's Rerank API** (docs.cohere.com/docs/rerank-2) or run an open-weights cross-encoder locally via SentenceTransformers using **BAAI/bge-reranker-v2-m3** (huggingface.co/BAAI/bge-reranker-v2-m3).
5. **Constrained Generation, Citations & Observability**
   Production hardening, cost tracking, and hallucination prevention
   Now wire up the generation layer with strict schema enforcement, refusal logic, and full request tracing so any bad answer can be debugged in seconds.
   - **What to build:**
     1. **Prompt Assembly & XML Tagging:** Format the Top-5 reranked chunks clearly using XML tags (<document id="chunk_42" source="runbook.md">...</document>) so the LLM can unambiguously distinguish instructions from untrusted data (mitigating indirect prompt injection).
     2. **Structured Output & Citation Verification:** Force the model to output structured JSON containing the response text and a list of cited chunk IDs. Write a post-processing validator in code that verifies every cited chunk_id actually exists in the 5 chunks provided to that request.
     3. **Observability & Telemetry Log:** Instrument every query to log a structured trace: original query, rewritten query, retrieved chunk IDs + RRF scores, reranker scores, prompt/completion token counts, total cost in USD, and latency breakdown per stage (Retrieval ms vs. Rerank ms vs. LLM TTFT).
   - **Direction & Sources:**
     - Use **Instructor** (python.useinstructor.com or js.useinstructor.com) or native **OpenAI / Anthropic Structured Outputs** to enforce strict schema validation and automatic retries when validation fails.
     - Integrate an open-source LLM observability tool like **Langfuse** (langfuse.com/docs, self-hostable via Docker) or

Arize Phoenix (docs.arize.com/phoenix) to visualize your multi-step traces and latency bottlenecks.

How to Present This to Employers in Your README

When an engineering manager or staff engineer reviews your repository, they will read your README.md before looking at your code. Structure your repository documentation around **engineering trade-offs and empirical metrics**:

- **Architecture Diagram:** Show both the async ingestion pipeline and the multi-stage query pipeline.
- **The Ablation Benchmark Table:** Include a real table showing your 30-question evaluation scores across four iterations:
    1. *Baseline Naive RAG (Fixed chunks + Vector only)*
    2. *+ Structure-Aware & Contextual Chunking*
    3. *+ Hybrid Search (Vector + BM25 RRF)*
    4. *+ Cross-Encoder Reranking*
- **Latency & Cost Profile:** State the p50/p95 latency and average cost per 1,000 queries (e.g., showing how retrieving 25 chunks and reranking down to 5 saves LLM input tokens compared to stuffing 20 raw chunks into the prompt).

Explore the interactive architecture diagram below to inspect how data flows through each stage of the **OpsPulse** pipeline, including the exact inputs, outputs, and failure modes prevented at every layer: